

Professor: Evandro

Assunto: Análise de Dados com R - Manipulação de Dados (dplyr)

Ambiente de Trabalho

Nesta aula utilizaremos as bibliotecas `dplyr` e `tidyr`, que fazem parte da biblioteca `tidyverse`, que é uma biblioteca específica do R Studio. Por este motivo, utilizaremos o R Studio online para compilar o código desta aula. Segue link abaixo. É necessário que o aluno vincule uma de suas contas de e-mail para ter acesso ao ambiente (Aqui, obviamente, escolha-se a opção/plano gratuito).

https://posit.cloud/content/yours?sort=name_asc

Transformação de Dados

A biblioteca `dplyr` apresenta 6 verbos de manipulação e transformação de dados que facilitam muito estas tarefas:

- **filter**, para filtrar linhas
- **select**, para selecionar colunas
- **arrange**, para ordenar os dados
- **mutate**, para criar novas colunas
- **group_by**, para agrupar dados
- **summarize**, para juntar valores dos dados agrupados

Veremos também a biblioteca `tidyr` para pivotar os dados.

Para instalar as bibliotecas rode os comandos abaixo. As bibliotecas `dplyr` e `tidyr` fazem parte da biblioteca `tidyverse`. A biblioteca `gapminder` é uma base de dados pronta que utilizaremos para teste.

```
> install.packages("tidyverse")  
> install.packages("gapminder")
```

Carregue as bibliotecas

```
> library(tidyverse)  
> library(gapminder)
```

Operador Pipe %>%

%>%: O operador pipe serve para concatenar verbos do dplyr. Basicamente ele direciona a saída de um comando do dplyr para outro comando dplyr.

Usaremos o pipe nos comandos a seguir:

Filtre linhas com filter()

filter() permite que você crie um subconjunto de observações com base em seus valores. O primeiro argumento é o dataframe. O segundo argumento e os subsequentes são as expressões que filtram o dataframe.

```
> # jeito tradicional
> gapminder[gapminder$year==1957, ]
# A tibble: 142 x 6
  country      continent  year lifeExp      pop gdpPercap
  <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
1 Afghanistan Asia      1957   30.3  9240934    821.
2 Albania     Europe    1957   59.3  1476505   1942.
3 Algeria     Africa    1957   45.7 10270856   3014.
4 Angola      Africa    1957   32.0  4561361   3828.
5 Argentina   Americas  1957   64.4 19610538   6857.
6 Australia   Oceania   1957   70.3  9712569  10950.
7 Austria     Europe    1957   67.5  6965860   8843.
8 Bahrain     Asia      1957   53.8   138655  11636.
9 Bangladesh  Asia      1957   39.3 51365468    662.
10 Belgium    Europe    1957   69.2  8989111   9715.
# ... with 132 more rows
```

```
> # jeito dplyr com pipe
> gapminder %>% filter(year==1957)
# A tibble: 142 x 6
  country      continent  year lifeExp      pop gdpPercap
  <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
1 Afghanistan Asia      1957   30.3  9240934    821.
2 Albania     Europe    1957   59.3  1476505   1942.
3 Algeria     Africa    1957   45.7 10270856   3014.
4 Angola      Africa    1957   32.0  4561361   3828.
5 Argentina   Americas  1957   64.4 19610538   6857.
6 Australia   Oceania   1957   70.3  9712569  10950.
7 Austria     Europe    1957   67.5  6965860   8843.
8 Bahrain     Asia      1957   53.8   138655  11636.
9 Bangladesh  Asia      1957   39.3 51365468    662.
10 Belgium    Europe    1957   69.2  8989111   9715.
# ... with 132 more rows
```

```
> # jeito dplyr sem pipe
> filter(gapminder, year==1957)
# A tibble: 142 x 6
  country      continent  year lifeExp      pop gdpPercap
  <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
1 Afghanistan Asia      1957   30.3  9240934    821.
2 Albania     Europe    1957   59.3  1476505   1942.
3 Algeria     Africa    1957   45.7 10270856   3014.
4 Angola      Africa    1957   32.0  4561361   3828.
5 Argentina   Americas  1957   64.4 19610538   6857.
6 Australia   Oceania   1957   70.3  9712569  10950.
7 Austria     Europe    1957   67.5  6965860   8843.
8 Bahrain     Asia      1957   53.8   138655  11636.
9 Bangladesh  Asia      1957   39.3 51365468    662.
10 Belgium    Europe    1957   69.2  8989111   9715.
# ... with 132 more rows
```

```
> ## filtrar o dataframe para China em 2002
> gapminder %>% filter(year==2002, country=='China')
# A tibble: 1 x 6
  country continent  year lifeExp      pop gdpPercap
  <fct>    <fct>    <int>  <dbl>    <int>    <dbl>
1 China   Asia      2002   72.0 1280400000   3119.
```

Selecione colunas com select()

`select()` permite que você selecione rapidamente um subconjunto útil usando operações baseadas nos nomes das variáveis.

```
> # selecionar somente país, ano e população
> gapminder %>% select(country, year, pop)
# A tibble: 1,704 x 3
  country      year      pop
  <fct>        <int>    <int>
1 Afghanistan 1952  8425333
2 Afghanistan 1957  9240934
3 Afghanistan 1962 10267083
4 Afghanistan 1967 11537966
5 Afghanistan 1972 13079460
6 Afghanistan 1977 14880372
7 Afghanistan 1982 12881816
8 Afghanistan 1987 13867957
9 Afghanistan 1992 16317921
10 Afghanistan 1997 22227415
# ... with 1,694 more rows
```

Combinando comandos

Selecionar linhas e colunas usando o filter e select com pipe.

```
> gapminder %>% filter(year==1957) %>% select(country, pop)
# A tibble: 142 x 2
  country      pop
  <fct>      <int>
1 Afghanistan 9240934
2 Albania      1476505
3 Algeria      10270856
4 Angola        4561361
5 Argentina    19610538
6 Australia     9712569
7 Austria       6965860
8 Bahrain       138655
9 Bangladesh   51365468
10 Belgium      8989111
# ... with 132 more rows
```

Note que a ordem é importante. Não dá para filtrar pela coluna ano se, no comando select anterior, você não escolheu a coluna ano para fazer parte do dataframe resultante.

```
> # selecionar país, e população no ano de 1957
> gapminder %>% select(country, pop) %>% filter(year==1957)
Error: Problem with `filter()` input `..1`.
i Input `..1` is `year == 1957`.
x object 'year' not found
Run `rlang::last_error()` to see where the error occurred.
```

Ordene linhas com arrange()

arrange() muda a ordem dos dados. É necessário um dataframe e uma ordem baseada nos nomes das coluna (ou expressões) para ordenar.

```
> # ordenar por expectativa de vida em ordem crescente
> gapminder %>% arrange(lifeExp)
# A tibble: 1,704 x 6
  country      continent  year lifeExp      pop gdpPercap
  <fct>      <fct>    <int>  <dbl>   <int>   <dbl>
1 Rwanda      Africa    1992   23.6  7290203    737.
2 Afghanistan Asia      1952   28.8  8425333    779.
3 Gambia      Africa    1952   30    284320    485.
4 Angola      Africa    1952   30.0  4232095   3521.
5 Sierra Leone Africa    1952   30.3  2143249    880.
6 Afghanistan Asia      1957   30.3  9240934    821.
7 Cambodia    Asia      1977   31.2  6978607    525.
8 Mozambique  Africa    1952   31.3  6446316    469.
9 Sierra Leone Africa    1957   31.6  2295678   1004.
10 Burkina Faso Africa    1952   32.0  4469979    543.
# ... with 1,694 more rows
```

```
> # ordenar por expectativa de vida em ordem decrescente
```

```
> gapminder %>% arrange(desc(lifeExp))
```

```
# A tibble: 1,704 x 6
```

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	Japan	Asia	2007	82.6	127467972	31656.
2	Hong Kong, China	Asia	2007	82.2	6980412	39725.
3	Japan	Asia	2002	82	127065841	28605.
4	Iceland	Europe	2007	81.8	301931	36181.
5	Switzerland	Europe	2007	81.7	7554661	37506.
6	Hong Kong, China	Asia	2002	81.5	6762476	30209.
7	Australia	Oceania	2007	81.2	20434176	34435.
8	Spain	Europe	2007	80.9	40448191	28821.
9	Sweden	Europe	2007	80.9	9031088	33860.
10	Israel	Asia	2007	80.7	6426679	25523.

```
# ... with 1,694 more rows
```

```
> # usando filter com arrange
```

```
> # filtrar o ano de 1957 e ordenar pela população em ordem decrescente
```

```
> gapminder %>% filter(year==1957) %>% arrange(desc(pop))
```

```
# A tibble: 142 x 6
```

	country	continent	year	lifeExp	pop	gdpPercap
	<fct>	<fct>	<int>	<dbl>	<int>	<dbl>
1	China	Asia	1957	50.5	637408000	576.
2	India	Asia	1957	40.2	409000000	590.
3	United States	Americas	1957	69.5	171984000	14847.
4	Japan	Asia	1957	65.5	91563009	4318.
5	Indonesia	Asia	1957	39.9	90124000	859.
6	Germany	Europe	1957	69.1	71019069	10188.
7	Brazil	Americas	1957	53.3	65551171	2487.
8	United Kingdom	Europe	1957	70.4	51430000	11283.
9	Bangladesh	Asia	1957	39.3	51365468	662.
10	Italy	Europe	1957	67.8	49182000	6249.

```
# ... with 132 more rows
```

Adicione novas variáveis com mutate()

Muitas vezes é útil adicionar novas colunas que são funções de colunas existentes. Esse é o trabalho de `mutate()`. Esta função sempre adiciona novas colunas no final do seu conjunto de dados.

```
> # criar a variável (lifeExpMonths=12*lifeExp)
> gapminder %>% mutate(lifeExpMonths=12*lifeExp)
# A tibble: 1,704 x 7
  country      continent year lifeExp      pop gdpPercap
lifeExpMonths
  <fct>        <fct>    <int>  <dbl>    <int>    <dbl>
<dbl>
1 Afghanistan Asia      1952   28.8  8425333    779.
346.
2 Afghanistan Asia      1957   30.3  9240934    821.
364.
3 Afghanistan Asia      1962   32.0 10267083    853.
384.
4 Afghanistan Asia      1967   34.0 11537966    836.
408.
5 Afghanistan Asia      1972   36.1 13079460    740.
433.
6 Afghanistan Asia      1977   38.4 14880372    786.
461.
7 Afghanistan Asia      1982   39.9 12881816    978.
478.
8 Afghanistan Asia      1987   40.8 13867957    852.
490.
9 Afghanistan Asia      1992   41.7 16317921    649.
500.
10 Afghanistan Asia      1997   41.8 22227415    635.
501.
# ... with 1,694 more rows
```

Crie estatísticas descritivas com summarize()

O `summarize()` agrupa o dataframe em uma única linha aplicando uma função a uma determinada coluna.

```
> # sumarizar a expectativa de vida mediana em 1957
> gapminder %>% filter(year==1957) %>%
+   summarize(medianLifeExp=median(lifeExp))
# A tibble: 1 x 1
  medianLifeExp
  <dbl>
1          48.4
```

```

> # sumarizar a média de expectativa de vida e o pib per
cápita máximo do ano de 1957
> gapminder %>% filter(year==1957) %>%
+       summarize(meanLifeExp = mean(lifeExp),
+       maxGdpPercap = max(gdpPercap))
# A tibble: 1 x 2
  meanLifeExp maxGdpPercap
    <dbl>         <dbl>
1      51.5      113523.

```

Agrupando dados com group_by() e summarize()

`summarize()` não é muito útil a menos que o emparelhemos com `group_by()`. Isso muda a unidade de análise do conjunto de dados completo para grupos individuais. Então, quando você usa os verbos `dplyr` em um dataframe agrupados, eles serão aplicados automaticamente "por grupo".

```

> # calcular a expectativa de vida média, o PIB per capta
máximo em cada continente
> # no ano de 1957
> gapminder %>% filter(year==1957) %>%
+       group_by(continent) %>%
+       summarize(meanLifeExp = mean(lifeExp),
+       maxGdpPercap = max(gdpPercap))
# A tibble: 5 x 3
  continent meanLifeExp maxGdpPercap
  <fct>         <dbl>         <dbl>
1 Africa      41.3           5487.
2 Americas    56.0          14847.
3 Asia        49.3          113523.
4 Europe      66.7          17909.
5 Oceania     70.3          12247.

```

```

> # calcular a expectativa de vida média, o PIB per capita
máximo em cada continente e por ano
> gapminder %>% group_by(continent, year) %>%
+   summarize(meanLifeExp = mean(lifeExp),
+             maxGdpPercap = max(gdpPercap))
`summarize()` has grouped output by 'continent'. You can
override using the `groups` argument.
# A tibble: 60 x 4
# Groups:   continent [5]
  continent year meanLifeExp maxGdpPercap
  <fct>      <int>      <dbl>      <dbl>
1 Africa    1952         39.1        4725.
2 Africa    1957         41.3        5487.
3 Africa    1962         43.3        6757.
4 Africa    1967         45.3       18773.
5 Africa    1972         47.5       21011.
6 Africa    1977         49.6       21951.
7 Africa    1982         51.6       17364.
8 Africa    1987         53.3       11864.
9 Africa    1992         53.6       13522.
10 Africa   1997         53.6       14723.
# ... with 50 more rows

```

Exercícios

Utilize o arquivo *questionario.txt*, disponível no Canvas da disciplina, para fazer os exercícios.

1. Escolha as amostras (linhas) apenas dos fumantes.
2. Calcular o índice de massa corporal (IMC) dos alunos e acrescentá-lo como uma coluna chamada `massa_corp` do data frame.

A fórmula do IMC é $IMC = \frac{peso}{altura^2}$, com peso em quilogramas e a altura em metros.

3. Qual o sexo e a turma do aluno com o maior peso?
4. Quais os alunos da turma B que são mais baixos que o aluno mais baixo da turma A?